DATA ENGINEERING & CLOUD · LESSON 3 OF 15

ETL versus ELT and Pipeline Reliability: Idempotency, Retries and Incremental Loads

Choose between transforming data before loading (ETL) and loading first then transforming (ELT), and build pipelines that survive reruns, retries and partial failures, using idempotent writes, bounded retries, incremental loads and reconciliation checks.

Gap-fill lesson · 2026 · Intermediate · 120 min

By the end of this lesson you can

- Explain when ETL is the right choice and when ELT is
- Make a load idempotent with overwrite-by-partition or merge-on-key
- Apply bounded retries with backoff and know what must not be retried
- Reconcile a load against its source with counts and totals

Datasets used

`daily_sales.csv`

ETL and ELT

ETL (extract, transform, load) cleans and reshapes data in a separate processing step, then loads the result. **ELT** (extract, load, transform) loads the raw data first, then transforms it inside the warehouse. The difference is where the work happens and what is kept.

Question	Favours ETL	Favours ELT
Must sensitive data be masked before it lands?	Yes	No, if the warehouse can restrict access
Is the warehouse elastic and cheap to compute?	Less relevant	Yes
Must raw data be kept for rebuilds?	Less often	Yes
Are many teams building different views of the same raw data?	No	Yes
Is the transformation complex and not SQL-friendly?	Yes	Harder

CASE STUDY **Masking before the warehouse**

A bank's card transactions include full card numbers. The warehouse is shared with many teams. The pipeline must replace the card numbers with a token before anything is written to the warehouse. That is an ETL requirement: the transformation happens before the load, so raw card numbers never land.

Most modern cloud teams use ELT for analytics and ETL for sensitive or complex steps. Both are valid; the choice is a decision about where the risk and the cost should sit.

Idempotency: the property that makes reruns safe

An operation is **idempotent** when running it again gives the same final result. Pipelines rerun all the time: after a failure, after a fix, or when a scheduler fires twice. A load that simply appends rows doubles the totals on every rerun. Two patterns make a load idempotent:

1. **Overwrite by partition.** Each run rewrites the partition it covers (for example, one day). Rerunning the same day replaces the same rows.
2. **Merge on a key.** Each row has a stable key; the load updates existing keys and inserts new ones.

Python

```python
import pandas as pd
import duckdb

con = duckdb.connect()
sales = pd.read_csv("daily_sales.csv", parse_dates=["date"])
con.register("sales", sales)
con.execute("CREATE TABLE fact_daily AS SELECT * FROM sales WHERE 1 = 0")

def load_partition(con, day):
    """Overwrite exactly one day's partition, so reruns cannot double it."""
    con.execute("DELETE FROM fact_daily WHERE date = ?", [day])
    con.execute("INSERT INTO fact_daily SELECT * FROM sales WHERE date = ?", [day])
    return con.execute("SELECT COUNT(*) FROM fact_daily WHERE date = ?", [day]).fetchone()
[0]

print("first run:", load_partition(con, "2025-01-06"))
print("rerun:    ", load_partition(con, "2025-01-06"))     # same result, not doubled
print("total rows:", con.execute("SELECT COUNT(*) FROM fact_daily").fetchone()[0])
```

OUTPUT

```
first run: 1
rerun:     1
total rows: 1
```

The rerun reports the same count, and the table holds one copy of each day. Compare that with a plain `INSERT` run twice, which would double the day.

Retries: bounded, backed off and selective

Some failures are temporary (a network timeout, a database that is briefly busy). Retrying them is correct. Other failures are permanent (a missing column, a bad credential) and retrying them wastes time and can hide the cause. A retry policy needs three rules:

- **Bounded:** a fixed number of attempts, then fail loudly.
- **Backed off:** wait longer between attempts (1, 2, 4 seconds), so the system can recover.
- **Selective:** retry only errors that can be temporary.

Python

```python
import time

def with_retries(fn, attempts=4, base_delay=0.01, retryable=(TimeoutError,
ConnectionError)):
    for i in range(attempts):
        try:
            return fn()
        except retryable:
            if i == attempts - 1:
                raise                                   # give up and surface the error
            time.sleep(base_delay * (2 ** i))
        # any other exception is not retried: it is a permanent error

calls = {"n": 0}
def flaky_load():
    calls["n"] += 1
    if calls["n"] < 3:
        raise TimeoutError("warehouse busy")
    return "loaded"

print(with_retries(flaky_load), "after", calls["n"], "attempts")
```

OUTPUT

```
loaded after 3 attempts
```

A schema error or a permission error is not in the `retryable` list, so it fails immediately with its real message. The retry is for the transient failure only.

::::warn Do not retry non-idempotent writes blindly Retrying a write that may have partly succeeded can duplicate it. Pair retries with idempotent loads (partition overwrite or merge on key), or with a check that asks whether the write already happened. ::::

Incremental loads and state

An incremental pipeline keeps a small piece of state: the watermark from the ingestion lesson, and the last successful partition. Store the state only after the data is committed, so a crash between the two leaves the state behind the data, and the next run safely reprocesses rather than skipping.

Reconciliation: prove the load is complete

A load that finishes without an error is not necessarily correct. **Reconciliation** compares the target with the source: row counts, sums of key measures, and distinct key counts, per partition.

Python

```python
src = sales[sales["date"] == "2025-01-06"]
tgt = con.execute("SELECT * FROM fact_daily WHERE date = '2025-01-06'").fetchdf()

checks = {
    "row count": (len(src), len(tgt)),
    "units total": (int(src["units_sold"].sum()), int(tgt["units_sold"].sum())),
}
for name, (expected, actual) in checks.items():
    status = "PASS" if expected == actual else "FAIL"
    print(f"{status}  {name}: source={expected} target={actual}")
```

OUTPUT

```
PASS  row count: source=1 target=1
PASS  units total: source=297 target=297
```

Run reconciliation after every load, store the results, and alert on any failure. A mismatch is the earliest and cheapest sign of a bad load.

A reliability checklist

- [ ] Each load is idempotent: a rerun gives the same result.
- [ ] Retries are bounded, backed off and limited to transient errors.
- [ ] State (watermark, last partition) advances only after a commit.
- [ ] Reconciliation runs after every load, with stored results.
- [ ] Failures alert a named owner with the partition and the error.
- [ ] A rerun procedure is written down and has been tested.

INTERVIEW CORNER **What makes a pipeline safe to rerun?**

Each run writes to a deterministic target: it overwrites the partition it owns or merges on a key, so the same input always produces the same output. Retries cover only transient errors, state advances after the commit, and reconciliation proves the result. Those four together are what make rerunning a routine action rather than a risk.

::::key What to remember - ETL transforms before loading; ELT loads raw data and transforms in the warehouse. Choose by risk, cost and rebuild needs. - Idempotency means a rerun gives the same result: overwrite partitions or merge on keys. - Retry only transient errors, with a limit and backoff. - Advance state only after commit, and reconcile every load against its source. ::::

Quick quiz

5 questions · tap an answer to check it

1. Why choose ETL for card numbers?

A It is faster

B The sensitive data must be masked before it lands in the shared warehouse

C ELT cannot store text

D It needs no testing

Answer: B. Transformation before loading protects raw sensitive data.

2. A load appends rows and is rerun after a timeout. What is the likely result?

A Nothing changes

B Rows are duplicated and totals are overstated

C The table is deleted

D The load is faster

Answer: B. Appends are not idempotent.

3. Which error should a retry policy retry?

A A missing column

B A wrong password

C A connection timeout

D A syntax error in the query

Answer: C. Only temporary failures are worth retrying.

4. What does reconciliation compare?

A The colours of the dashboard

B Counts and totals in the target against the source, per partition

C The names of the engineers

D The file sizes

Answer: B. Reconciliation proves completeness and correctness.

5. When should the watermark or last-partition state be advanced?

A Before the load starts

B After the data is committed successfully

C Every minute

D Never

Answer: B. State behind the data is safe; state ahead of the data loses rows.

EXERCISE **Make a load safe to rerun**

Write a function `load_day(con, day)` that loads one day of `daily_sales.csv` into a target table using a merge on the date (replace the day if it exists), then run it three times for two different days. Show the row count after each run and a reconciliation PASS for each day.

Show solution (worked answer)

Hints: delete the day, then insert, inside one transaction (`BEGIN` and `COMMIT`). Reconcile the units total for that day against the source file. The totals should be identical after every rerun.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.